

CLASE 2

Proyecto profesional y entorno Django

Preparar una base reproducible antes de desarrollar funcionalidades



```
graph TD; A[Repositorio] --> B[Entorno]; B --> C[Proyecto];
```

Repositorio
→
Entorno
→
Proyecto

El problema: funciona en mi computador

- Django fue instalado globalmente.
- La carpeta del entorno terminó en Git.
- Nadie registró las dependencias.
- El equipo copió comandos de otra terminal.

**Un entorno no verificado
convierte el inicio en
una fuente de errores.**

¿Cuál de estos problemas puede afectar a todo el equipo?

Todos pueden afectar la reproducción del proyecto.

Hoy dejaremos una base que otra persona pueda reproducir

OBJETIVO DE APRENDIZAJE

- **Preparar un proyecto Django para EcoEnergy utilizando Git**
- **Un entorno virtual aislado**
- **Dependencias registradas**
- **Verificaciones antes del primer commit.**

Al finalizar podrás...

- 01 Confirmar que Python se ejecuta desde .venv.
- 02 Crear y comprobar el proyecto config sin errores.
- 03 Registrar solo archivos necesarios en un commit limpio.

El orden evita registrar archivos locales por accidente

01

Clonar y revisar

02

Crear .gitignore

03

**Preparar .venv y
Django**

04

**Verificar y
confirmar**

Regla de trabajo:
ejecutar → observar → explicar → continuar.

¿por qué .gitignore debe existir antes de crear .venv?

Comenzamos desde el repositorio y observamos su estado

```
git clone <URL_DEL_REPOSITORIO>
cd ecoenergy-backend
git status
git branch --show-current
```

Antes de modificar

- Confirma la carpeta correcta.
- Revisa la rama activa.
- Detecta cambios previos.

El comando **git branch --show-current** imprime únicamente el nombre de la rama en la que te encuentras trabajando en ese momento

El repositorio conserva el proyecto, no el estado de cada equipo

SÍ SE VERSIONA

- Código fuente y configuración compartida.
- requirements.txt y README.md.
- .gitignore y .env.example.

NO SE VERSIONA

- .venv y archivos de caché.
- Secretos reales del entorno.
- Configuración personal del editor.

Git debe permitir reconstruir el proyecto sin publicar archivos locales ni secretos.

.gitignore protege el repositorio desde el primer minuto

```
# .gitignore
.venv/
__pycache__/
*.py[cod]
.env
db.sqlite3
.vscode/
```

Decisión inicial

- Ignorar entorno y caché.
- Excluir secretos locales.
- Evitar datos de desarrollo.

El comando de activación depende de la terminal

GIT BASH

```
python -m venv .venv  
  
source .venv/Scripts/activate
```

POWERSHELL

```
python -m venv .venv  
  
.\.venv\Scripts\Activate.ps1
```

Si hay problema de **Política de Ejecución**

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

esa línea **autoriza temporalmente a PowerShell** para que te deje ejecutar scripts en tu computadora.

**No cambies comandos al azar:
Identifica primero qué terminal estás utilizando.**

Práctica: crear y comprobar .venv

SITUACIÓN

Cada estudiante debe demostrar que el Python activo pertenece al repositorio EcoEnergy.

1
Crear .venv

2
Activar según
terminal

3
Consultar
sys.executable

4
Revisar git status

Evidencia: ruta dentro de .venv y carpeta .venv ausente de git status.

```
python -c "import sys; print(sys.executable)";
```

Es extremadamente útil para **saber si tu entorno virtual .venv se activó bien**.

- **Si NO está activado**, te mostrará la ruta global (ej. \Programs\Python\Python312\python.exe).
- **Si SÍ está activado con éxito**, te mostrará la ruta interna de tu proyecto (ej. \.venv\Scripts\python.exe).

Instalamos Django usando el Python ya verificado

```
python -m pip install Django  
python -m django --version  
python -m pip show Django
```

Verificaciones

- pip pertenece al Python activo.
- Django responde con una versión.
- Se observa su ubicación.

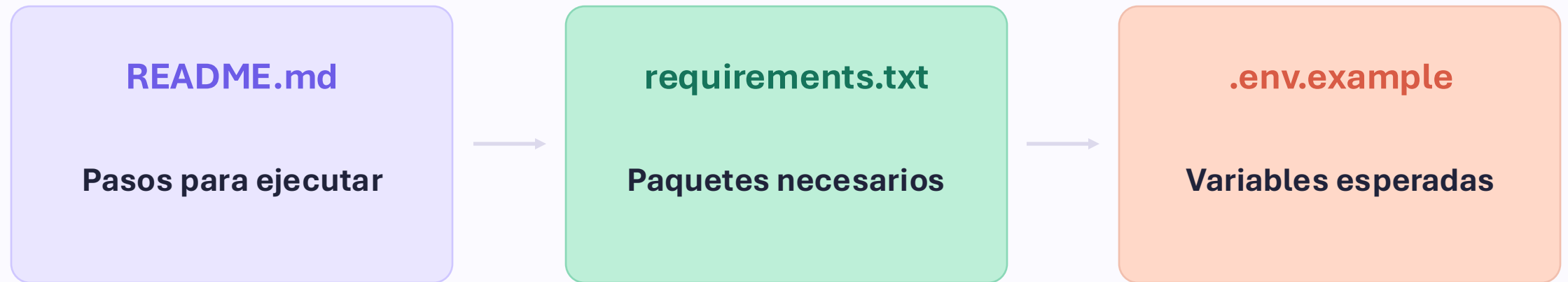
requirements.txt convierte una instalación local en una instrucción reproducible

```
python -m pip freeze > requirements.txt  
  
python -m pip install -r requirements.txt
```

Dos direcciones

- freeze registra dependencias.
- install -r las reconstruye.
- El archivo sí se versiona.

Cuatro archivos explican cómo colaborar sin compartir secretos



.gitignore completa el contrato al **excluir archivos locales y valores sensibles.**

Creamos el proyecto Django dentro de la raíz del repositorio

```
python -m django startproject config .  
  
python manage.py check
```

Observa el punto final

- config contiene la configuración.
- manage.py queda en la raíz.
- check valida la configuración.

La estructura separa ejecución y configuración



Todavía no creamos una app: primero comprobamos que la base funciona.

Levantamos el servidor y comprobamos desde dos lugares

01

Ejecutar check

`python manage.py check`

02

Iniciar runserver

`python manage.py runserver`

03

Abrir
127.0.0.1:8000

04

Detener con
Ctrl+C

Evidencia: terminal sin errores y página inicial de Django visible.

Un error útil se lee desde la última causa concreta

```
ModuleNotFoundError: No module named  
'django'
```

```
Error: That port is already in use.
```

```
can't open file 'manage.py'
```

**Cada mensaje apunta a
una verificación
diferente.**

¿Qué comprobarías primero en cada caso?

Entorno, puerto y carpeta actual.

La IA nos ayuda a construir un README útil

Ayúdame a redactar el README .md inicial de mi proyecto Backend desarrollado con Python y Django.
Mi ruta de trabajo es: **[Proyecto Integrado / EcoEnergy]**.

Incluye:

- Descripción y objetivo.
- Requisitos previos.
- Clonación del repositorio.
- Creación y activación de .venv.
- Instalación desde requirements.txt.
- Comandos de verificación.
- Estado actual y próximos pasos.

No inventes versiones, URL ni funcionalidades. Si falta información, pregúntame antes de redactar.

Tu responsabilidad

- Entregar información real del proyecto.
- Revisar que los comandos correspondan a Git Bash.
- Ejecutar y comprobar las instrucciones.
- Eliminar datos personales o secretos.
- Mantener actualizado el README.

La IA propone la documentación; el estudiante comprueba que realmente funciona.

Revisamos exactamente qué entrará al primer commit

```
git status
git diff
git add .
git diff --cached
git status
```

Antes de confirmar

- Nada sensible aparece.
- .venv sigue ignorado.
- El conjunto es coherente.

git diff: Muestra los cambios en tu carpeta de trabajo que **no** han sido guardados con git add.

git diff --cached: Muestra los cambios que **sí** han sido guardados con git add

El commit registra una base funcional y explicable

```
git commit -m "chore: preparar proyecto Django"
git status
git log -1 --oneline
```

Definición de limpio

- El commit describe una intención.
- Git confirma el registro.
- El árbol queda sin cambios.

En Git, **chore** es un tipo de prefijo usado en los mensajes de confirmación (**commit messages**) bajo la especificación de Conventional Commits para indicar tareas de mantenimiento o rutina que **no modifican el código de producción ni agregan nuevas funciones**.

Desafío: otro equipo debe reproducir tu base

SITUACIÓN

En parejas, una persona revisa el repositorio de la otra sin copiar su carpeta `.venv`.

1

Revisar archivos
raíz

2

Seguir README

3

Ejecutar check

4

Registrar una
mejora

Evidencia: lista breve con una instrucción clara, una omisión detectada y el resultado de check.

Una base profesional se puede verificar y reconstruir

TICKET DE SALIDA

¿Qué comprobación evita instalar dependencias en el Python equivocado y qué archivo permite reconstruirlas?

Responde brevemente

1. Comprobación
2. Archivo
3. Justificación

PRÓXIMA CLASE

Clase 3: HTTP, URLs y Views · convertir una ruta en una respuesta Django.